Reasoning LLMs

Foundations · the DeepSeek R1 Paper · Working with Them

Lucas Soares · O'Reilly Live Training

Agenda

1. Foundations — what reasoning LLMs are, the 2026 landscape, when & how to use them

Agenda

1. Foundations — what reasoning LLMs are, the 2026 landscape, when & how to use them

2. The DeepSeek R1 Paper — an interactive walkthrough, and we rebuild it in code

Agenda

1. Foundations — what reasoning LLMs are, the 2026 landscape, when & how to use them

2. The DeepSeek R1 Paper — an interactive walkthrough, and we rebuild it in code

3. Working with Reasoning Models — the practical API & engineering half

01

Foundations

What they are, the landscape, when & how to use them

What do we mean by "reasoning"?

Recall — look something up:

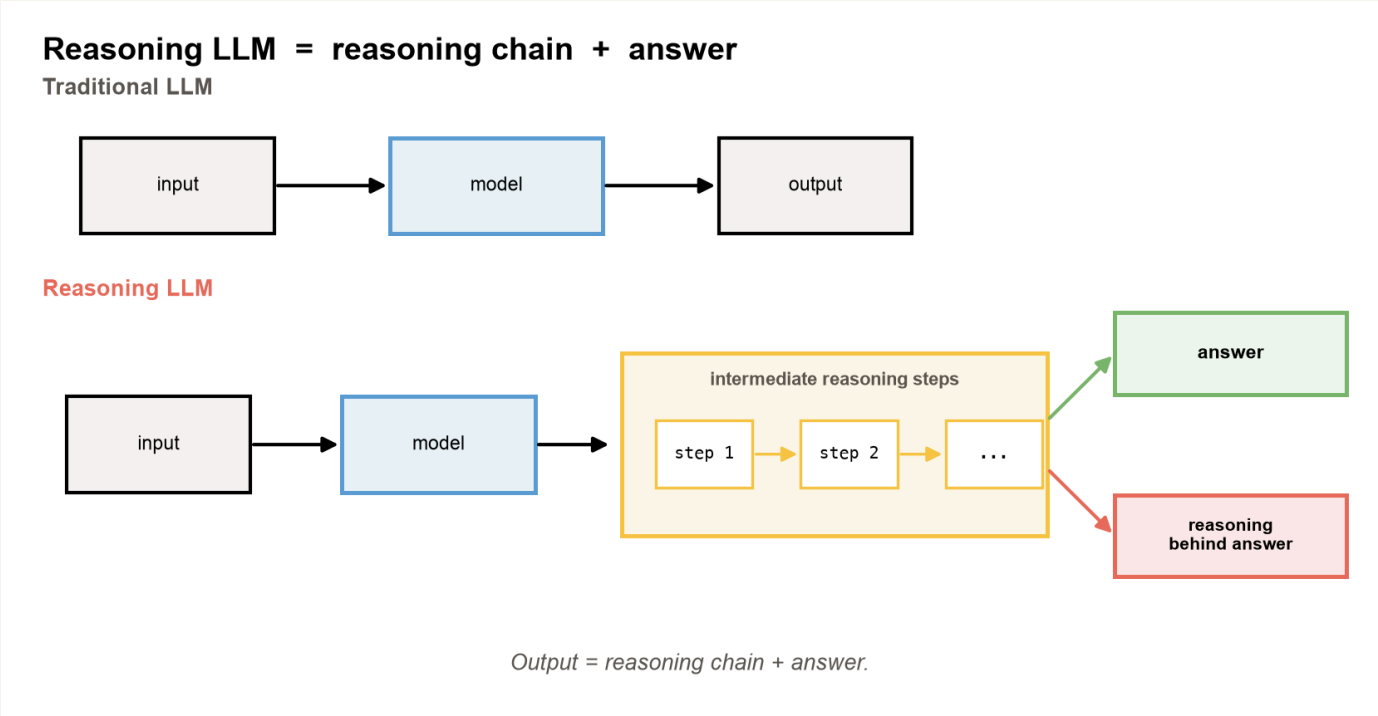
> "What's the course code for Stanford's Transformers class?" → CME 295

Reasoning — work it out:

> "A bear was born in 2020. How old in 2025?" → 2025 − 2020 = **5**

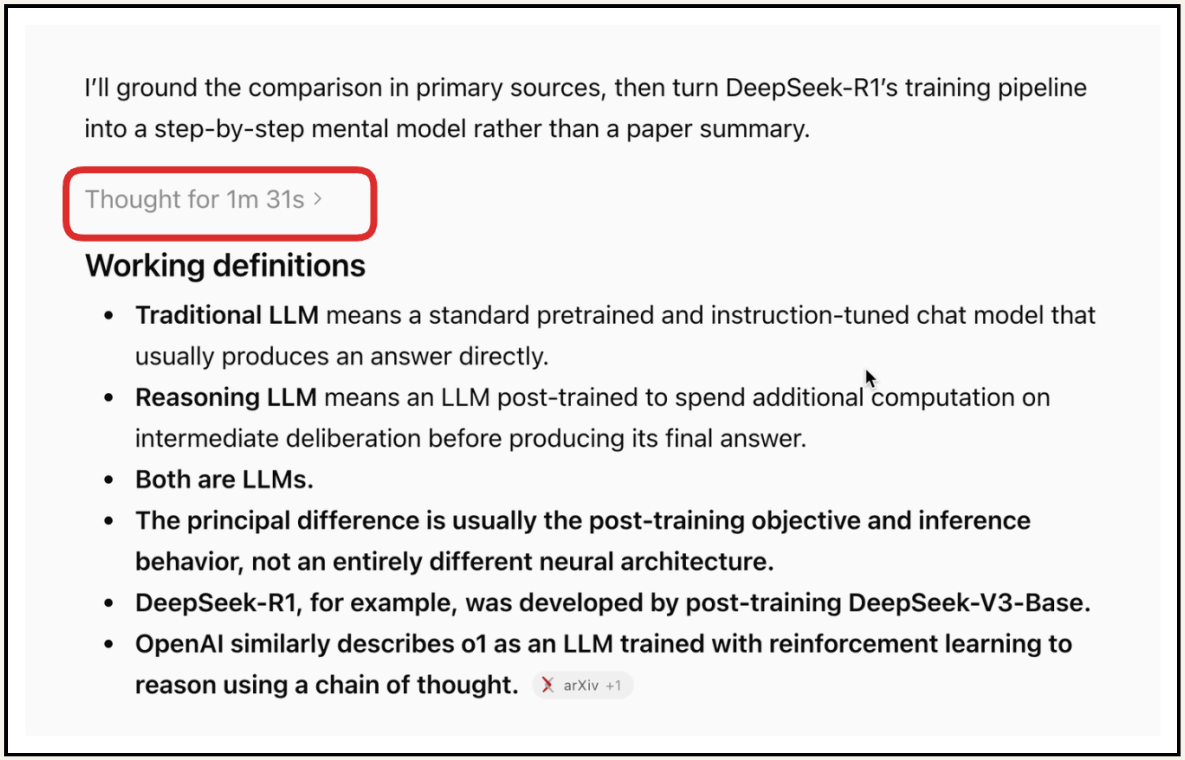
Reasoning = solving a problem through a **multi-step process**, not a single lookup.

What is a reasoning LLM?



The output contract changes: **Output = reasoning chain + answer.**

You've already seen this — for real



I'll ground the comparison in primary sources, then turn DeepSeek-R1's training pipeline into a step-by-step mental model rather than a paper summary.

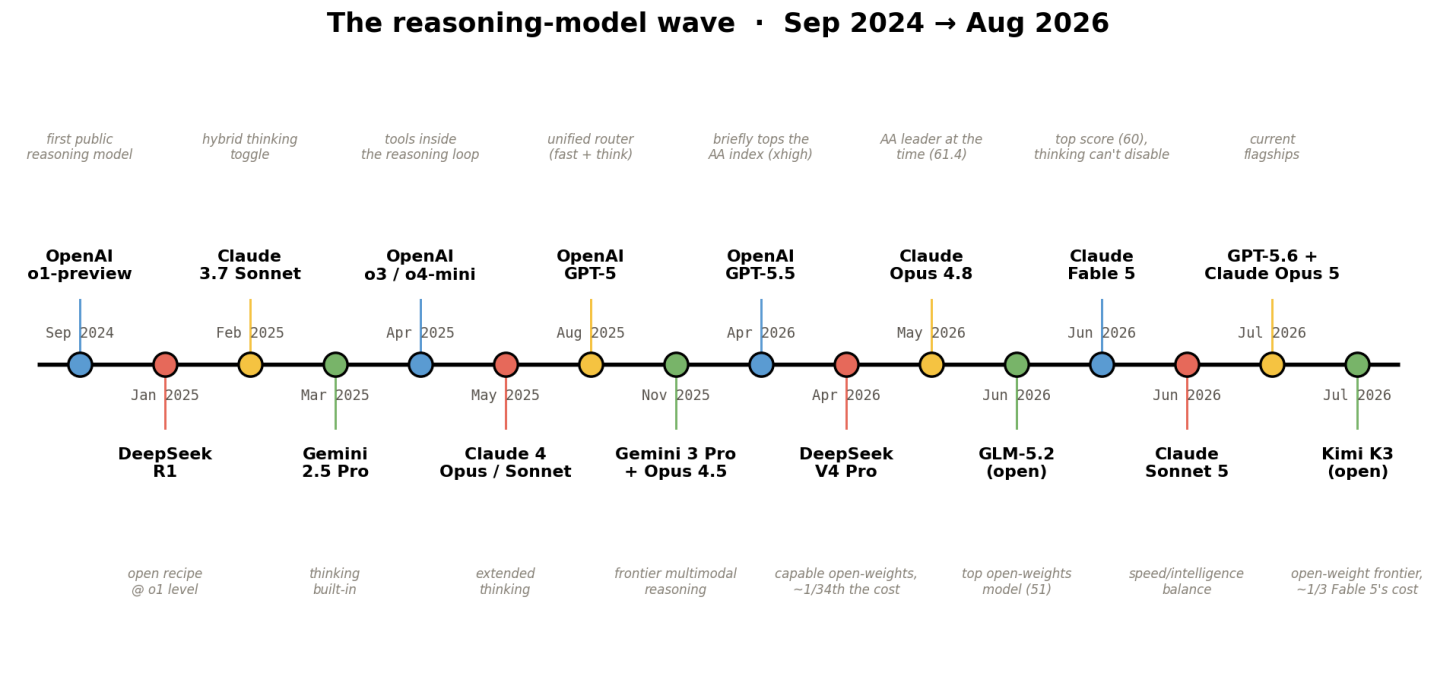
Thought for 1m 31s ›

Working definitions

- **Traditional LLM** means a standard pretrained and instruction-tuned chat model that usually produces an answer directly.
- **Reasoning LLM** means an LLM post-trained to spend additional computation on intermediate deliberation before producing its final answer.
- **Both are LLMs.**
- **The principal difference is usually the post-training objective and inference behavior, not an entirely different neural architecture.**
- **DeepSeek-R1, for example, was developed by post-training DeepSeek-V3-Base.**
- **OpenAI similarly describes o1 as an LLM trained with reinforcement learning to reason using a chain of thought.** arXiv +1

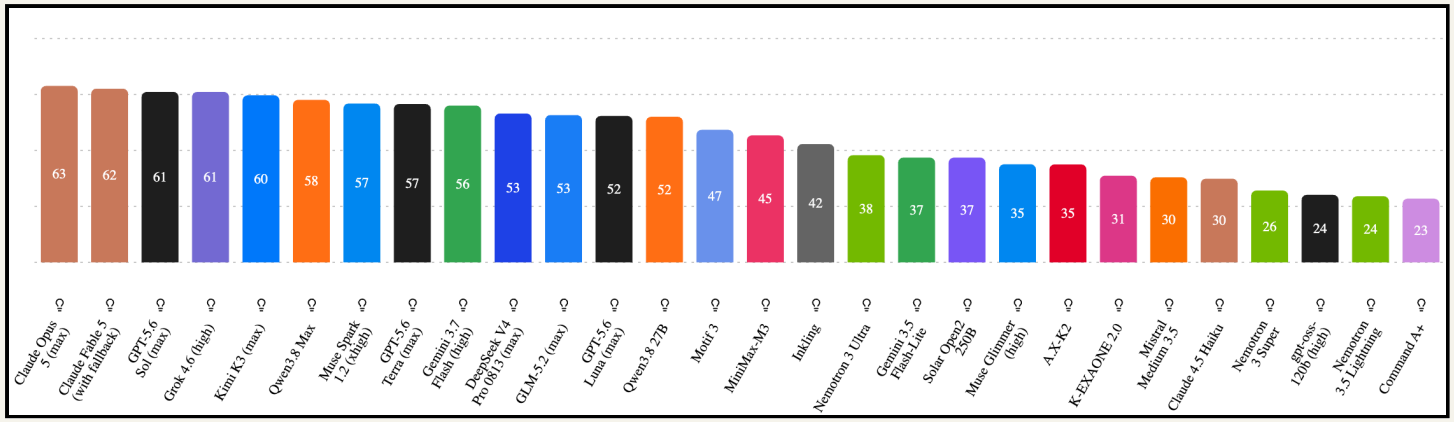
A real ChatGPT answer: it **"Thought for 1m 31s"** before writing a word. The thinking pill is the tell — ChatGPT, Claude, Gemini, DeepSeek all show it.

How we got here



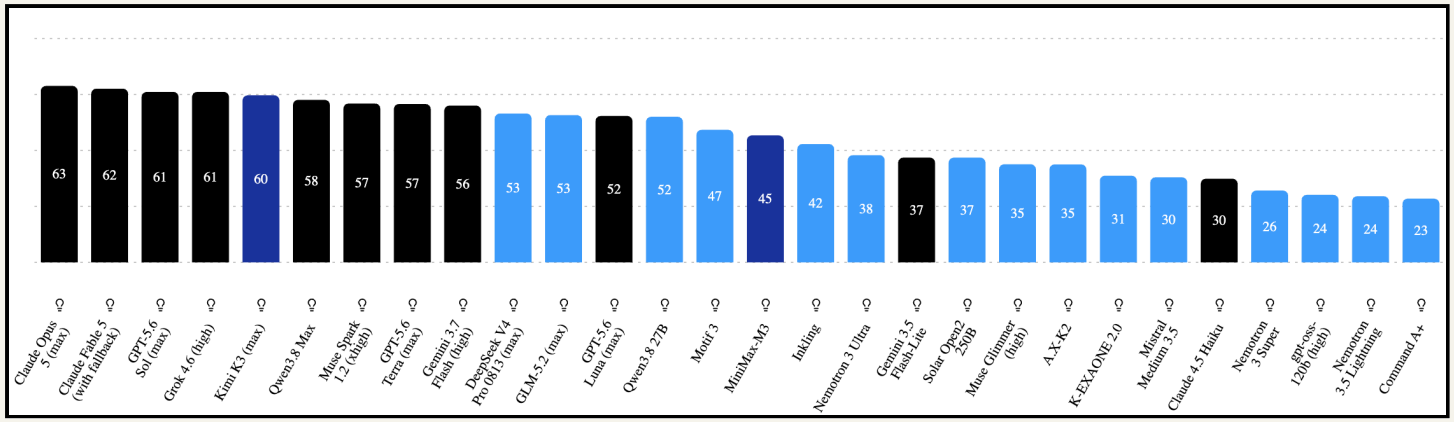
o1-preview kicked it off (Sep 2024). DeepSeek-R1 (Jan 2025) made the **recipe public**. By 2026 every flagship reasons.

Where the models rank today



Artificial Analysis Intelligence Index v4.1.1 — the top of the board is a cluster of reasoning models (⚡ = reasoning). *Live: artificialanalysis.ai/models*

The gap is closing



■ Proprietary ■ open-weights — open models (blue) now sit **right among the proprietary leaders**. Months behind, not years.

Source: artificialanalysis.ai

What actually matters when you pick one

1 · Intelligence / reasoning quality

composite score — note it's agentic-weighted

2 · Tool-use & agentic ability NEW

now the largest slice of "intelligence"

3 · Reliability & instruction-following NEW

pass@1, low hallucination

4 · Cost — per *task*, not per token

reasoning emits far more billed tokens

5 · Speed — split in two

output tokens/sec vs latency (TTFT)

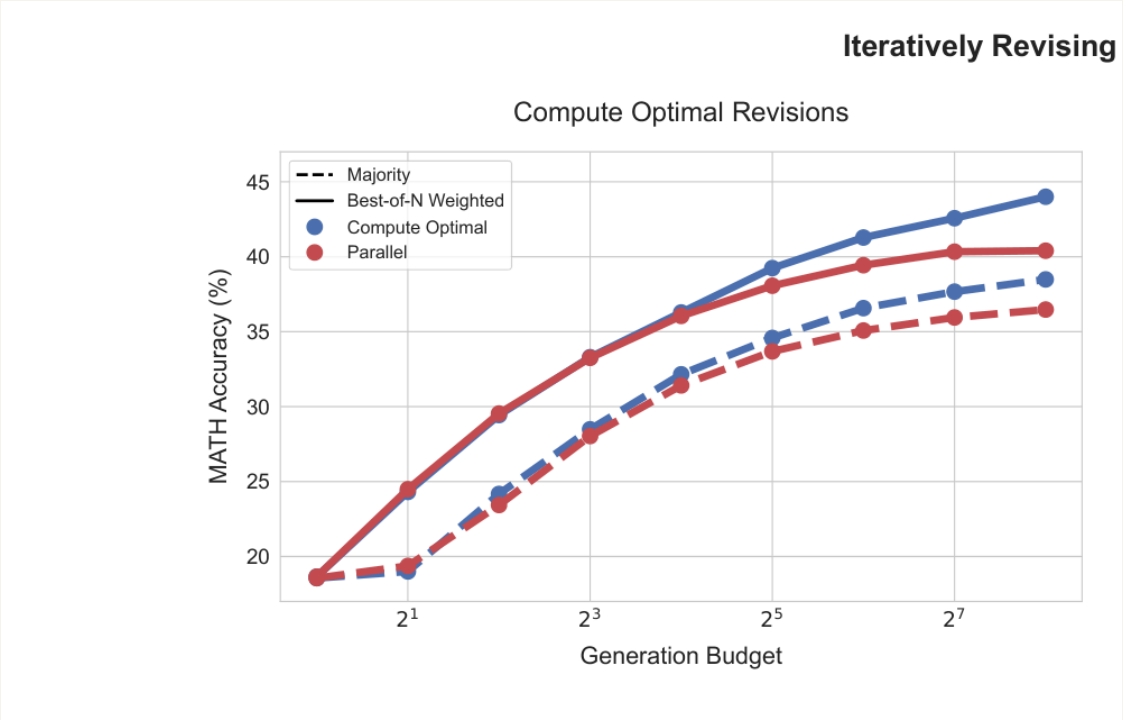
6 · Context window

RAG / long-document workloads

Updated from the old "quality · price · knowledge · speed · context" list: **tool-use** and **reliability** now dominate, and "built-in knowledge" folds into #1/#3 as retrieval becomes the default.

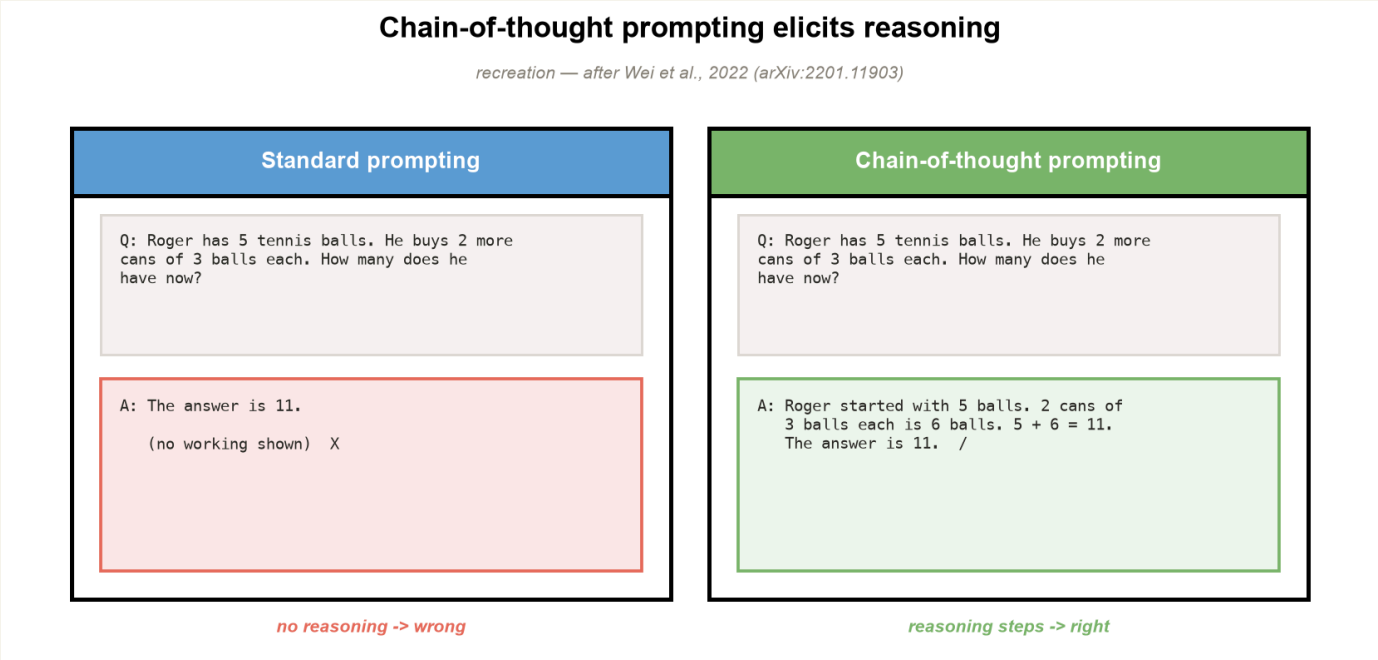
Adapted from the Artificial Analysis Intelligence Index methodology v4.1.1 (agentic 34% · coding 24% · sci-reasoning 24% · general 18%, scored pass@1). source

Why does thinking longer help?



Every generated token is another forward pass — reasoning tokens **buy compute** before the model commits.

The simplest version of the trick



Chain-of-thought: show the working, get the answer right. Scaled up, this *is* the core idea. *(after Wei et al., 2022 · arXiv:2201.11903)*

Two reasons a reasoning chain helps

1. **Decomposition** — break a problem the model has never seen into sub-problems it *has*.
2. **More compute** — more tokens before the answer = more computation spent on it.

> Extended chains and self-correction ("wait, that's wrong…") **emerge from RL**, not just from prompting.

When to use one — and when not to

Strong fit → reach for it

- Ambiguous, multi-step problems
- Long / dense document analysis
- Agentic planning & strategy
- Multi-file coding & debugging
- Visual reasoning · LLM-as-judge · hard math

Poor fit → use a plain model

- Fact lookup & retrieval
- Classification & routing
- Format conversion
- Latency-sensitive UX
- Long-output generation (cost)

Pattern: **reasoning model plans, cheaper model executes.** Default: *"when in doubt, respond directly"* — don't turn thinking on reflexively.

Synthesized from OpenAI, Anthropic & Google reasoning guides. Hard cost/latency numbers: our reproducibility notebook.

How to prompt a reasoning model — 1/3: universal principles

- Keep it **short, direct, goal-oriented** — these models reward clarity, not verbosity
- **Don't hand-write the reasoning** — say *"think thoroughly,"* not *"think step by step"*
- State **success criteria + hard constraints** up front (role → goal → constraints)
- Set depth with the **effort/thinking dial**, not with prompt padding
- **Zero-shot first**; add a few structured examples only if needed

The reasoning is now the model's job. Your job is a crisp brief.

OpenAI reasoning best-practices · Anthropic extended-thinking tips · Gemini 3 guide.

How to prompt — 2/3: what a good prompt looks like

The shape

Role — who the model is

Goal — the outcome, not the steps

Constraints — hard rules & scope

Success criteria — what "done" means

Direct · goal-first · zero-shot.
Set *depth* with the effort dial, not the prompt.

Applied

```
# role
You are a senior tax analyst.
# goal
Find every deduction this 2024 US filing qualifies
for.
# constraints
- cite the IRS form + line for each
- flag anything ambiguous; don't guess
# success
A table an accountant could file from.
```

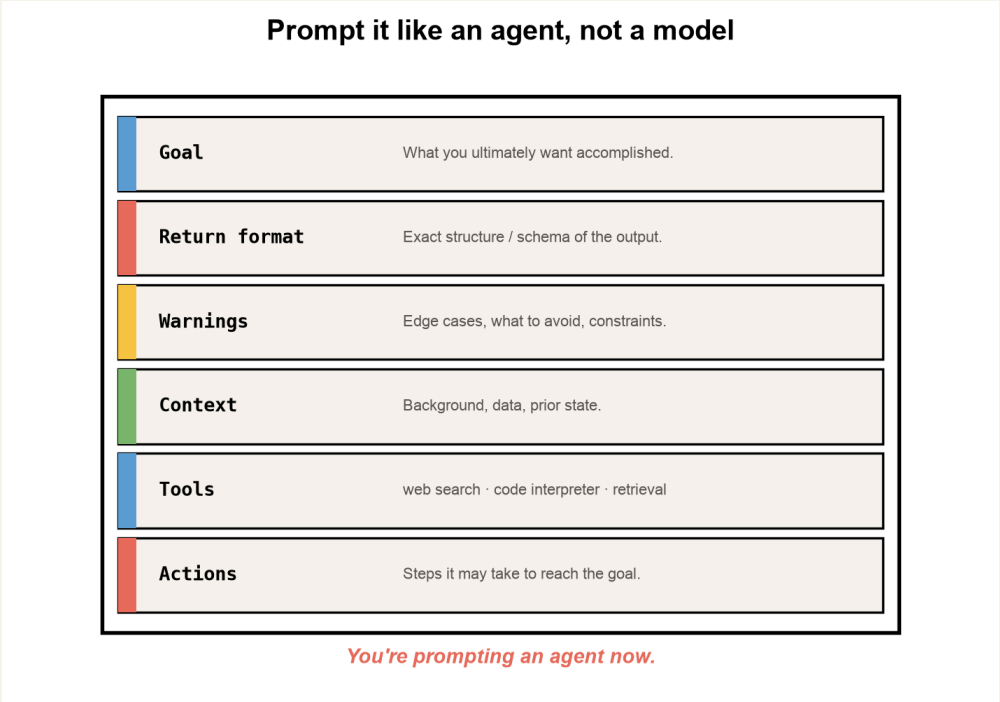
Notice what's *absent*: no "think step by step", no hand-written reasoning — that's the model's job.

How to prompt — 3/3: anti-patterns

✗ Don't	✓ Do instead
Hand-write "think step by step"	Let it reason — say "think thoroughly"
Contradict yourself ("detailed" + "concise")	One clear directive; separate concerns
Lower Gemini 3's temperature to reason	Keep temperature at 1.0
"Always double-check" on newest Opus	Trust its self-verification; don't nag
Max effort on every simple task	Match effort to task complexity

Each "don't" costs tokens, latency, or quality for no gain. (API params & deprecated knobs → Section 3.)

You're not prompting a model — you're prompting an agent



Goal · return format · context · tools · actions. Modern reasoning systems route and act — this framing still holds. *(confirmed against 2026 provider agent guides)*

02

The DeepSeek R1 Paper

An interactive walkthrough — and we rebuild it in code

arxiv.org/pdf/2501.12948

Why this paper matters

Open weights + an **open recipe** for o1-level reasoning.

DEMO — two ways to train an LLM, overlaid

`guide/training-overlay.html`

Open the overlay: the **same pretrained base** → the paths diverge. **Top:** Llama-2 chat (RLHF + PPO). **Bottom:** DeepSeek-R1 (GRPO + verifiable rewards).

Recap: the fork is at SFT (human QA vs long CoT); the deepest change is the **reward** — learned preference model + PPO critic *vs* rule-based verifiable rewards + GRPO with no critic.

Interactive walkthrough of the paper

📖 Open the guide → `guide/reasoning-training-guide.html`

A self-contained, interactive artifact that merges a **pedagogical explanation** with the **paper itself** — step through R1-Zero vs R1, the multi-stage pipeline, GRPO, and the embedded paper figures.

The rest of this section walks the same pipeline — and we **rebuild each stage in code**.

Notebook: `notebooks/01_build_a_reasoning_model_r1_from_scratch.ipynb` — one toy task (1-digit addition), one metric (pass-rate) climbing through every stage.

How do you train a model to *think*?

You don't write the reasoning for it. You **incentivize** it:

- Show a few examples of **thinking through** a problem (cold-start)
- Then **reward** the behavior you want with RL:
  - did it get the answer **right**? (accuracy)
  - did it use the **`<think>` markers**? (format)
  - did it **stay in one language**? (consistency)

> No human writes the chains. The model discovers them.

The trick: verifiable rewards

For math and code you can **check the answer automatically** — no learned reward model, no human rater in the loop.

- Math → compare to the parsed ground-truth number
- Code → run the **unit tests**

> A cheap, unhackable signal is what makes pure-RL reasoning possible.

R1-Zero vs R1

R1-Zero

Pure RL on a base model, no SFT. Works — but ugly, language-mixed outputs.

R1

Add a small cold-start SFT + a second SFT pass. Same skill, readable.

The "aha moment"

Table 2 | An interesting "aha moment" of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to
Response: `<think>` To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\cdots$ $\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2$. Rearrange to isolate the inner square root term: $(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$ $\ldots$ Wait, wait. Wait. That's an aha moment I can flag here. Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$ We started with the equation: $\sqrt{a - \sqrt{a + x}} = x$ First, let's square both sides: $a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$ Next, I could square both sides again, treating the equation: $\cdots$ $\ldots$

Accuracy curve

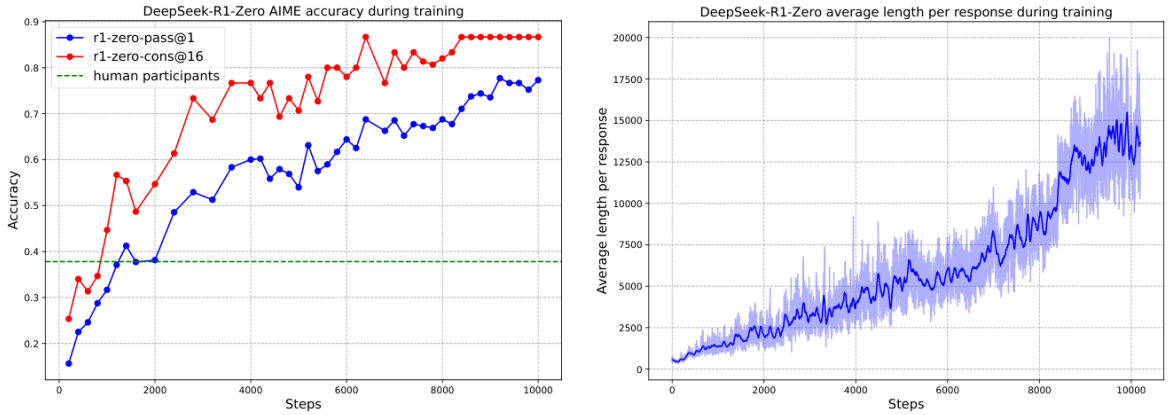


Figure 1 | (a) AIME accuracy of DeepSeek-R1-Zero during training. AIME takes a mathematical problem as input and a number as output, illustrated in Table 32. Pass@1 and Cons@16 are described in Supplementary D.1. The baseline is the average score achieved by human participants in the AIME competition. (b) The average response length of DeepSeek-R1-Zero on the training set during the RL process. DeepSeek-R1-Zero naturally learns to solve reasoning tasks with more thinking time. Note that a training step refers to a single policy update operation.

DEMO — a scratchpad beats a direct answer

`notebooks/01_build_a_reasoning_model_r1_from_scratch.ipynb` · §0 Foundations

Two tiny models on 1-digit addition — one answers **directly**, one gets a **scratchpad**. The scratchpad wins, measured on your laptop.

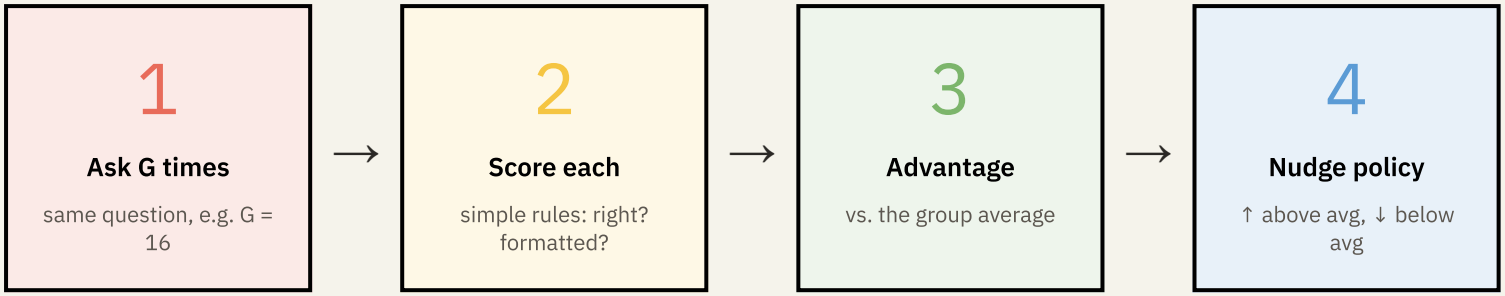
Recap: intermediate tokens aren't decoration — they're *computation*. That single result is why reasoning models exist.

The 5-stage recipe



Base → cold-start SFT → reasoning RL (GRPO) → rejection-sampling SFT → distillation. **Our notebook runs all five on a toy task.**

GRPO — the 30-second intuition



No value net. No human rater. The **group is the baseline**.

Why no critic?

PPO (classic RL)

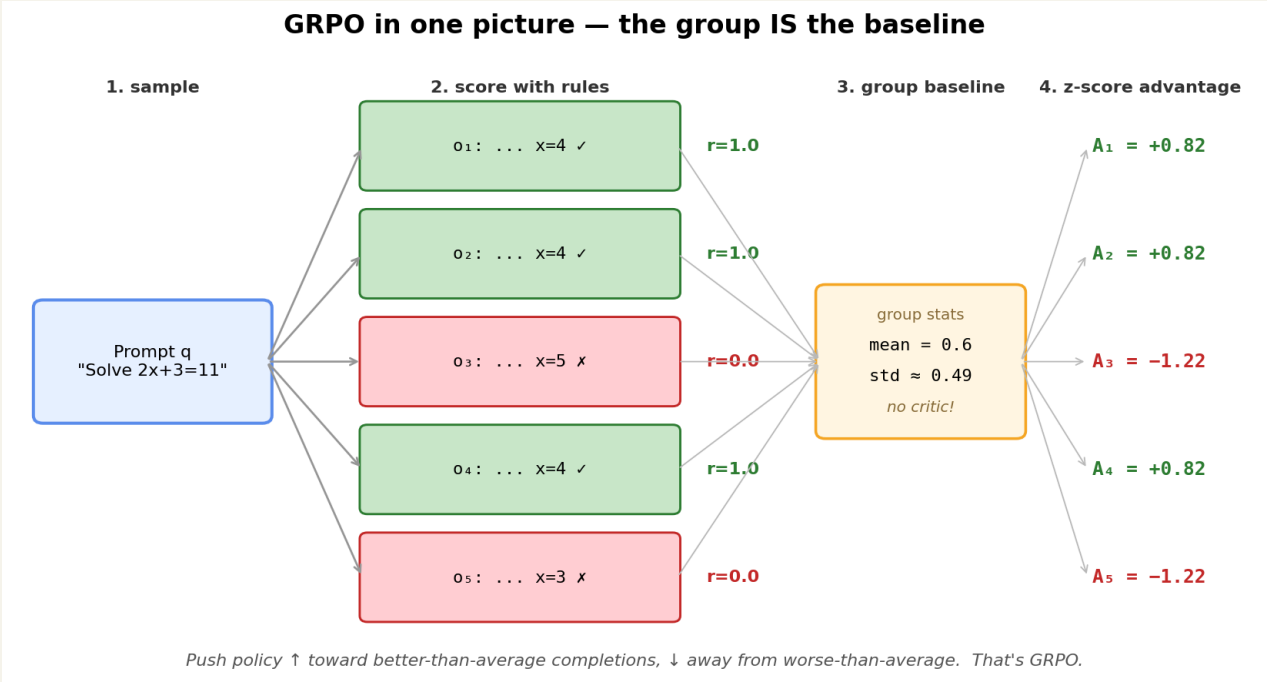
- **Baseline:** learned value model (extra NN)
- **Networks trained:** policy + value
- **Memory:** ~2×
- **Reward source:** learned reward model (usually)

GRPO (DeepSeek)

- **Baseline:** mean reward of the group
- **Networks trained:** policy only
- **Memory:** ~1×
- **Reward source:** rule-based (correct? formatted?)

Half the moving parts. No reward hacking from a learned critic.

GRPO in one picture



One prompt → G rollouts → rule-based rewards → z-score advantages.

The formal objective

For each question q, GRPO samples a group of outputs $\{o_1, o_2, \cdots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_θ by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right), \tag{1}$$

$$\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \tag{2}$$

where π_{ref} is a reference policy, ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \cdots, r_G\})}{\text{std}(\{r_1, r_2, \cdots, r_G\})}. \tag{3}$$

Clipped PPO-style update — but the advantage $\hat{A}$ is just the **z-score of the group rewards**. That's the whole trick.

DEMO — cold-start SFT → GRPO from scratch

`01_build_a_reasoning_model_r1_from_scratch.ipynb` · §3 cold-start · §4 GRPO

Teach the …… format, then reward correctness. No critic, no value network, no human labels.

Recap: cold-start locks the shape (answers still mostly wrong); GRPO makes the pass-rate *climb* — the group is the baseline.

Distillation results

Table 15 | Comparison of DeepSeek-R1 distilled models and other comparable models on reasoning-related benchmarks. Numbers in bold denote the performance is statistically significant (t−test with $p < 0.01$).

Model	AIME 2024		MATH	GPQA Diamond	LiveCode Bench	CodeForces
	pass@1	cons@64	pass@1	pass@1	pass@1	rating
GPT-4o-0513	9.3	13.4	74.6	49.9	32.9	759
Claude-3.5-Sonnet-1022	16.0	26.7	78.3	65.0	38.9	717
DeepSeek-R1-Distill-Qwen-1.5B	28.9	52.7	83.9	33.8	16.9	954
DeepSeek-R1-Distill-Qwen-7B	55.5	83.3	92.8	49.1	37.6	1189
DeepSeek-R1-Distill-Qwen-14B	69.7	80.0	93.9	59.1	53.1	1481
DeepSeek-R1-Distill-Qwen-32B	**72.6**	83.3	94.3	62.1	57.2	1691
DeepSeek-R1-Distill-Llama-8B	50.4	80.0	89.1	49.0	39.6	1205
DeepSeek-R1-Distill-Llama-70B	70.0	**86.7**	**94.5**	**65.2**	**57.5**	**1633**

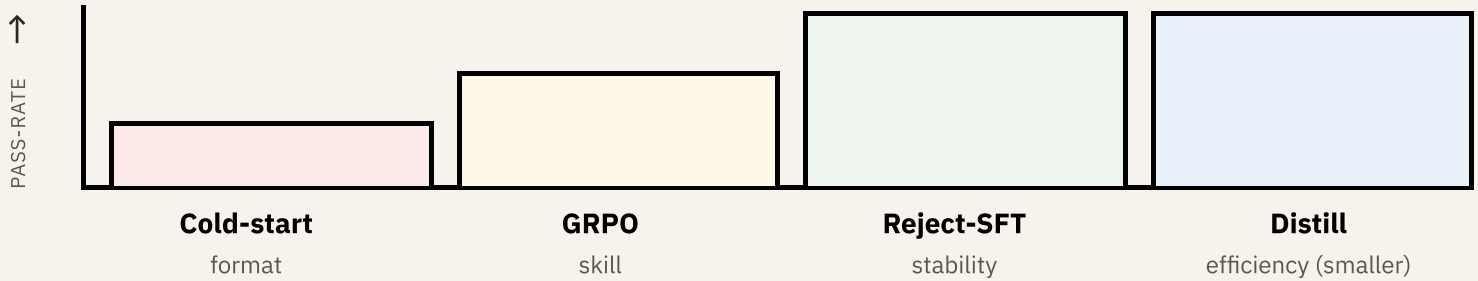
DEMO — amplify & compress

`01_build_a_reasoning_model_r1_from_scratch.ipynb` · §5 rejection-SFT · §6 distillation

Use the RL'd model as a **data factory** (generate → keep correct → fine-tune), then pour the skill into a **~7× smaller student**.

Recap: the pass-rate jumps again with no new training signal — just better data — and survives compression.

The payoff — one metric, the whole recipe



(Exact numbers print live in the notebook — that's the demo.)

This is still how it's done

The same recipe — **GRPO + RL from verifiable rewards** — is how 2026 frontier reasoning models (GPT-5.6, Claude Opus 5, Gemini 3.x) are trained.

> Scaled up massively. **Not replaced.**

03

Working with Reasoning Models

The practical API & engineering half

One fact drives every decision: the model spends hidden tokens thinking before it answers.

1 · The depth dial — one idea, three knobs

OpenAI	Anthropic	Google
`reasoning={"effort": ...}` none · low · medium · high · xhigh · max	`thinking={"type":"adaptive"}` + `output_config={"effort": ...}`	`thinking_level` minimal · low · medium · high

- Higher effort = more thinking tokens = better on hard tasks, slower + pricier; models scale **down** on easy inputs automatically
- Anthropic separates *whether* it thinks (`thinking`) from *how hard* (`effort`) — don't confuse them
- `budget_tokens` is **deprecated / 400** on newest Claude → use adaptive + effort
- `max_tokens` is the only **hard** ceiling; effort/budget are soft guidance

OpenAI reasoning guide · Anthropic thinking docs · Gemini thinking docs.

2 · Streaming: visible vs summarized

- You **never get the raw chain of thought** — only an opt-in *summary*
- OpenAI `reasoning.summary` (auto/concise/detailed) · Anthropic `display` (summarized/omitted) · Gemini `thinking_summaries`
- Streaming order is always: **thinking deltas** → **signature** → **answer text**
- `display: "omitted"` cuts time-to-first-answer-token but **not** cost
- Summaries come from a *separate* model; the answering model never sees them

```
content_block_delta(thinking_delta…) → signature_delta → content_block_delta(text_delta…)
```

3 · Reasoning tokens are billed as output tokens

The single most important cost fact — **hidden thinking still costs money.**

- Read it: OpenAI `reasoning_tokens` · Anthropic `thinking_tokens` · Gemini `total_thought_tokens`
- You pay for the **full** thinking even when you only see a short summary
- Reserve headroom (OpenAI suggests **~25k**) or a truncated run bills you for thinking with **no answer**
- Budget = f(effort level, task complexity) — **measure, then tune down**

A response that hits `max_output_tokens` mid-thought comes back `incomplete` — and you're still charged.

4 · Multi-turn: carry the reasoning back

Reasoning state does **not** follow the conversation automatically.

- **Anthropic:** echo every `thinking` block back *unchanged* (incl. `redacted_thinking`) during tool use — edit/reorder/drop → **400**
- **OpenAI:** `previous_response_id` (stateful) *or* replay `encrypted_content` (stateless/ZDR); `reasoning.context` picks scope
- **Gemini:** resend `thought` signatures verbatim in stateless mode
- Chat Completions drops reasoning items → prefer the **Responses API** for agentic loops

```
second = responses.create(model="gpt-5.6", previous_response_id=first.id, input="Now patch it.")
```

5 · Tool use with reasoning

- A tool loop is **one assistant turn**: model reasons → calls tools → answers
- **Anthropic:** pick thinking mode per turn (can't toggle mid-turn); **forced tool choice needs *adaptive* thinking**
- **Interleaved thinking** = reason *between* tool calls (adaptive = automatic; manual = beta header)
- **OpenAI:** echo reasoning items + function-call items back every hop; mind the `phase` field
- Production pattern: **reasoning model plans, cheap model executes**

thinking → tool → **thinking** → tool → answer (interleaved beats one-shot for long agentic runs)

6 · Structured outputs / JSON

- OpenAI Structured Outputs (`json_schema`, `strict: true`) **works** with reasoning models
- Strict schema requires: all fields `required`, `additionalProperties: false`
- Biggest break = **truncation**: thinking eats the budget, JSON stops mid-object → give it headroom
- Anthropic path is **tool-use schemas** (Pydantic in this course); forced tool choice needs adaptive thinking
- Keep schemas **byte-stable** so they stay in the prompt cache

Guard every structured call against `incomplete` / parse failure.

7 · Prompt caching interactions

Caching is where reasoning config quietly wrecks your hit rate.

- Cache hits need **byte-identical prefixes**; OpenAI cached input **≈ 0.1×** cost
- **Anthropic renders thinking config/effort into the prompt** → changing it **breaks the cache**
- Lock **one** effort/budget per cached conversation; don't tune it mid-thread
- During tool loops, thinking blocks cache *with* tool results (then bill as input on read)
- Long reasoning runs outlive the 5-min cache → use the **1-hour TTL**

8 · Latency & throughput

- To go faster: **drop effort first** — it shrinks thinking + answer together
- **Route by task:** reasoning model for hard/ambiguous, fast model for well-defined
- Stream, and use `display:"omitted"` to start the answer sooner (cost unchanged)
- Budgets > ~32k, or `max_tokens` > 21,333 → **stream and/or batch** to dodge timeouts
- Parallelize independent calls; spend high effort only where it pays

Latency ↔ effort is measurable — see the reproducibility notebook.

9 · Common pitfalls (the 400 club)

- **Truncation** is #1 — thinking exhausts the budget → incomplete output, still billed
- Newest Claude **rejects custom `temperature` / `top_p` / `top_k`** and any response **prefill**
- Never **edit, reorder, or drop** thinking blocks (incl. `redacted_thinking`) → 400
- Don't say *"think step by step"* — over-prompting CoT hurts; be direct & zero-shot
- OpenAI: **developer** (not system) messages; Anthropic: migrate off `budget_tokens`

10 · Production pre-flight checklist

- □ Right tool for the task?
- □ Token headroom reserved (~25k)?
- □ State handling correct per provider?
- □ Streaming + timeout strategy?
- □ Structured output guarded vs truncation?
- □ Effort chosen & **pinned**?
- □ Cost instrumented (log reasoning tokens)?
- □ Tool-loop mode set per turn?
- □ Cache byte-stable, config fixed?
- □ Prompts direct, zero-shot, no CoT padding?

Model choice → effort pinned → headroom → cost logged → state echoed → cache stable.

DEMO — provider reasoning controls, hands-on

`provider_openai_thinking_parameters.ipynb` · `provider_anthropic_extended_thinking.ipynb`

Dial `reasoning.effort` across levels; drive Claude adaptive/extended thinking. Watch tokens, latency, and answer quality move together.

Recap: same model, dial the depth to the task — the knob is now universal; the off-switch is disappearing.

DEMO — cheap vs flagship, reproducibly

`notebooks/02_reproducibility_cheap_vs_flagship.ipynb`

Fix a **seed**, match a flagship with a **cheaper** model, and **count the cost** — the engineering half of "which model do I ship?"

Recap: the expensive model isn't always the right call — measure per-task cost, not per-token price.

Recap — the three movements

- **Foundations** — output = reasoning + answer; reach for it on hard, ambiguous, agentic work
- **The R1 paper** — cold-start SFT → GRPO on verifiable rewards → reject-SFT → distill; we rebuilt it
- **Working with them** — pin effort, budget for hidden tokens, echo reasoning state, guard truncation

> The recipe is public and the knobs are in your hands. Now go ship one.

Questions?

DeepSeek R1 paper — arxiv.org/pdf/2501.12948 · interactive guide: `guide/reasoning-training-guide.html`

Notebooks: `01_build_a_reasoning_model_r1_from_scratch` · `02_reproducibility_cheap_vs_flagship` · provider deep-dives

Current model rankings/pricing: artificialanalysis.ai/models

Lucas Soares · O'Reilly Live Training